

Inventory API

Inventory management API for units of measure, categories, item types, dynamic attributes, attribute options, and inventory items.

Base URL

```
/api/auth/inventory
```

All endpoints require authentication and should be called with the authenticated user's credentials.

1. Units of Measure

Units define how inventory quantities are measured.

List Units

```
GET /units
```

Example:

```
GET /api/auth/inventory/units
```

Create Unit

```
POST /units
```

Request:

```
{
  "code": "PC",
  "name": "Piece",
  "symbol": "pc",
  "decimal_places": 0
}
```

Get Unit

```
GET /units/{unit}
```

Example:

```
GET /api/auth/inventory/units/1
```

Update Unit

```
PATCH /units/{unit}
```

Request:

```
{
  "name": "Pieces",
  "symbol": "pcs"
}
```

Activate Unit

```
POST /units/{unit}/activate
```

Deactivate Unit

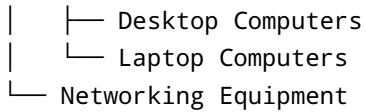
```
POST /units/{unit}/deactivate
```

2. Item Categories

Categories organize inventory items hierarchically.

Example:

```
IT Equipment
├─ Computers
```



List Categories

GET /categories

Create Category

POST /categories

Request:

```
{
  "parent_id": null,
  "code": "IT",
  "name": "IT Equipment",
  "description": "Information technology equipment."
}
```

Child category:

```
{
  "parent_id": 1,
  "code": "IT-COMPUTERS",
  "name": "Computers",
  "description": "Desktop and laptop computers."
}
```

Get Category

GET /categories/{category}

Update Category

PATCH /categories/{category}

Example:

```
{  
  "name": "Computer Equipment"  
}
```

A category cannot be assigned to itself or one of its descendants.

Delete Category

```
DELETE /categories/{category}
```

Category Tree

Returns the hierarchical category structure.

```
GET /categories/tree
```

Activate Category

```
POST /categories/{category}/activate
```

Deactivate Category

```
POST /categories/{category}/deactivate
```

3. Item Types

Item types determine how an item behaves.

Examples:

```
COMPUTER  
OFFICE_SUPPLY  
FURNITURE
```

BUILDING
LAND

Tracking Types

none
lot
serial

Example:

```
{
  "code": "COMPUTER",
  "name": "Computer Equipment",
  "description": "Computer equipment.",
  "tracking_type": "serial",
  "is_asset": true,
  "is_composite": true,
  "is_active": true
}
```

List Item Types

GET /item-types

Create Item Type

POST /item-types

Example:

```
{
  "code": "COMPUTER",
  "name": "Computer Equipment",
  "description": "Computer equipment.",
  "tracking_type": "serial",
  "is_asset": true,
  "is_composite": true
}
```

Get Item Type

```
GET /item-types/{itemType}
```

Update Item Type

```
PATCH /item-types/{itemType}
```

Activate Item Type

```
POST /item-types/{itemType}/activate
```

Deactivate Item Type

```
POST /item-types/{itemType}/deactivate
```

4. Attribute Definitions

Attribute definitions describe configurable fields that can be attached to item types.

Examples:

```
Brand  
Model  
Processor  
RAM  
Storage  
Operating System  
Color  
Material
```

Data Types

Supported attribute data types:

```
text
textarea
integer
decimal
boolean
date
datetime
select
multiselect
```

List Attributes

```
GET /attributes
```

Create Attribute

```
POST /attributes
```

Example:

```
{
  "code": "brand",
  "name": "Brand",
  "data_type": "select",
  "description": "Equipment manufacturer.",
  "is_required": true,
  "is_active": true,
  "sort_order": 1,
  "validation_rules": null,
  "default_value": null
}
```

Get Attribute

```
GET /attributes/{attribute}
```

Update Attribute

```
PATCH /attributes/{attribute}
```

Example:

```
{
  "name": "Manufacturer",
  "description": "Manufacturer of the equipment."
}
```

Activate Attribute

```
POST /attributes/{attribute}/activate
```

Deactivate Attribute

```
POST /attributes/{attribute}/deactivate
```

5. Attribute Options

Attribute options are used by `select` and `multiselect` attributes.

For example, the `brand` attribute can have:

```
Dell
HP
Lenovo
Acer
```

List Attribute Options

Options are scoped to an attribute.

```
GET /attributes/{attribute}/options
```

Example:

```
GET /api/auth/inventory/attributes/1/options
```

Create Attribute Option

```
POST /attributes/{attribute}/options
```

Request:

```
{
  "value": "dell",
  "label": "Dell",
  "sort_order": 1,
  "is_active": true
}
```

Get Attribute Option

```
GET /attribute-options/{option}
```

Update Attribute Option

```
PATCH /attribute-options/{option}
```

Example:

```
{
  "label": "Dell Technologies"
}
```

Delete Attribute Option

```
DELETE /attribute-options/{option}
```

Activate Attribute Option

```
POST /attribute-options/{option}/activate
```

Deactivate Attribute Option

```
POST /attribute-options/{option}/deactivate
```

6. Item Type Attributes

Item type attributes determine which dynamic attributes are available for a particular item type.

For example:

```
COMPUTER  
  
Brand  
Model  
Processor  
RAM  
Storage  
Operating System
```

while:

```
FURNITURE  
  
Brand  
Material  
Color  
Dimensions
```

The same attribute definition can therefore be reused by multiple item types.

List Item Type Attributes

```
GET /item-types/{itemType}/attributes
```

Example:

```
GET /api/auth/inventory/item-types/1/attributes
```

Assign Attribute to Item Type

```
POST /item-types/{itemType}/attributes
```

Request:

```
{
  "attribute_definition_id": 1,
  "is_required": true,
  "sort_order": 1
}
```

Update Item Type Attribute

```
PATCH /item-types/{itemType}/attributes/{itemTypeAttribute}
```

Example:

```
{
  "is_required": false,
  "sort_order": 2
}
```

Remove Attribute from Item Type

```
DELETE /item-types/{itemType}/attributes/{itemTypeAttribute}
```

7. Dynamic Form Metadata

The metadata endpoint returns the attributes required to construct an item's dynamic form.

```
GET /item-types/{itemType}/attributes/metadata
```

Example:

```
GET /api/auth/inventory/item-types/1/attributes/metadata
```

Example response:

```
{
  "item_type": {
    "id": 1,
    "code": "COMPUTER"
  },
  "attributes": [
    {
      "id": 1,
      "code": "brand",
      "name": "Brand",
      "data_type": "select",
      "required": true,
      "options": [
        {
          "value": "dell",
          "label": "Dell"
        },
        {
          "value": "hp",
          "label": "HP"
        }
      ]
    },
    {
      "id": 2,
      "code": "ram",
      "name": "RAM",
      "data_type": "integer",
      "required": false
    }
  ]
}
```

The frontend can use this response to dynamically construct the item form.

8. Items

Items are the actual inventory records.

The `items` table contains only the fixed/common fields.

Dynamic attributes are stored separately.

List Items

```
GET /items
```

Create Item

```
POST /items
```

Example:

```
{
  "name": "Dell Precision 3680",
  "item_type_id": 1,
  "category_id": 2,
  "unit_of_measure_id": 1,
  "barcode": "123456789",
  "reorder_level": 1,
  "reorder_quantity": 2,
  "attributes": {
    "brand": "Dell",
    "model": "Precision 3680",
    "processor": "Intel i7-14700",
    "ram": 32
  }
}
```

The `item_code` is generated by the backend.

Example:

```
ITM-000001
```

Do not send `item_code` when creating an item.

Get Item

```
GET /items/{item}
```

Example response:

```
{
  "data": {
    "id": 100,
    "item_code": "ITM-000100",
    "name": "Dell Precision 3680",
    "item_type": {
      "id": 1,
      "code": "COMPUTER",
      "name": "Computer Equipment"
    },
    "category": {
      "id": 2,
      "name": "IT Equipment"
    },
    "unit": {
      "code": "PC",
      "name": "Piece"
    },
    "tracking_type": "serial",
    "attributes": {
      "brand": "Dell",
      "model": "Precision 3680",
      "processor": "Intel i7-14700",
      "ram": 32
    }
  }
}
```

Update Item

```
PATCH /items/{item}
```

Example:

```
{
  "name": "Dell Precision 3680 Workstation",
}
```

```
"reorder_level": 2
}
```

Dynamic attributes can be updated independently through the attributes endpoint.

Delete / Archive Item

```
DELETE /items/{item}
```

Items use soft deletes, so deletion archives the record rather than permanently removing it.

Restore Item

```
POST /items/{item}/restore
```

Activate Item

```
POST /items/{item}/activate
```

Deactivate Item

```
POST /items/{item}/deactivate
```

9. Item Dynamic Attributes

Dynamic attribute values are stored separately from the `items` table.

For example:

```
Item
├─ Dell Precision 3680
│   │
│   ├─ brand = Dell
│   ├─ model = Precision 3680
│   ├─ processor = Intel i7-14700
│   └─ ram = 32
```

Get Item Attributes

```
GET /items/{item}/attributes
```

Example:

```
GET /api/auth/inventory/items/100/attributes
```

Example response:

```
{
  "data": {
    "brand": "Dell",
    "model": "Precision 3680",
    "processor": "Intel i7-14700",
    "ram": 32
  }
}
```

Update Item Attributes

```
PATCH /items/{item}/attributes
```

Example:

```
{
  "attributes": {
    "brand": "HP",
    "model": "Z2 G9",
    "processor": "Intel i7",
    "ram": 64
  }
}
```

The backend validates these values against the item's `ItemType`.

For example, if `ram` is defined as:

```
data_type = integer
```

then:

```
{
  "ram": 32
}
```

is valid, while:

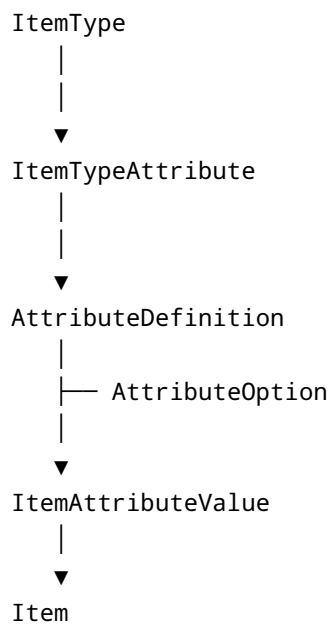
```
{
  "ram": "thirty-two"
}
```

is rejected.

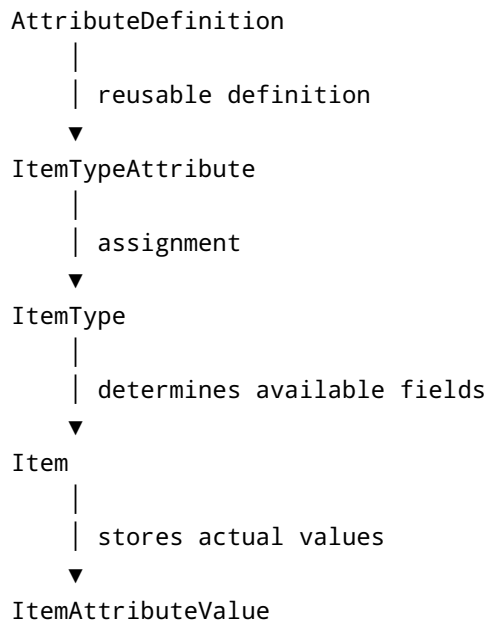
For a `select` attribute, the submitted value must exist in the attribute's active options.

10. Dynamic Attribute Architecture

The relationship between the entities is:



More specifically:



This means an attribute such as `brand` can be reused:

```
Computer
└─ Brand

Printer
└─ Brand

Furniture
└─ Brand
```

without creating three separate attribute definitions.

11. Typical Workflow

Step 1 — Create an Attribute

```
POST /attributes
```

```
{
  "code": "brand",
  "name": "Brand",
```

```
"data_type": "select",  
"is_required": true  
}
```

Step 2 — Add Options

```
POST /attributes/1/options
```

```
{  
  "value": "dell",  
  "label": "Dell"  
}
```

```
POST /attributes/1/options
```

```
{  
  "value": "hp",  
  "label": "HP"  
}
```

Step 3 — Assign Attribute to Item Type

```
POST /item-types/1/attributes
```

```
{  
  "attribute_definition_id": 1,  
  "is_required": true,  
  "sort_order": 1  
}
```

Step 4 — Request Metadata

```
GET /item-types/1/attributes/metadata
```

The frontend now knows that the Computer form needs a `brand` field and that it is a required `select`.

Step 5 — Create the Item

```
POST /items
```

```
{
  "name": "Dell Precision 3680",
  "item_type_id": 1,
  "category_id": 2,
  "unit_of_measure_id": 1,
  "attributes": {
    "brand": "dell"
  }
}
```

The backend generates:

```
ITM-000001
```

and persists the dynamic attribute value separately.

12. Endpoint Summary

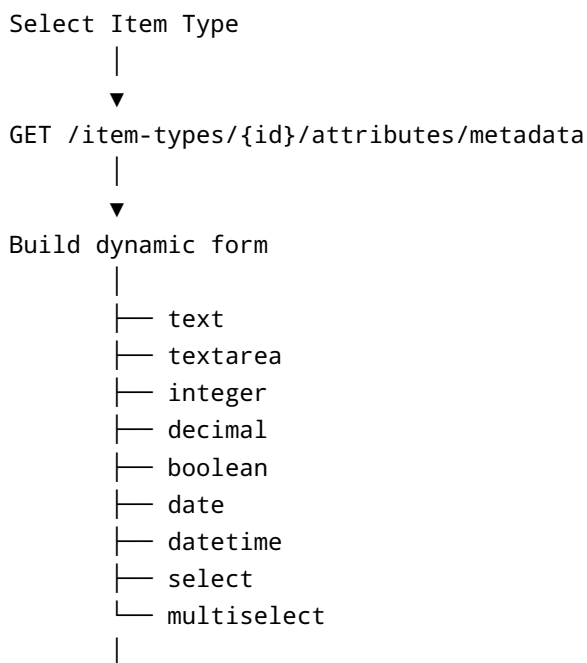
Method	Endpoint	Purpose
GET	/units	List units
POST	/units	Create unit
GET	/units/{unit}	Get unit
PATCH	/units/{unit}	Update unit
POST	/units/{unit}/activate	Activate unit
POST	/units/{unit}/deactivate	Deactivate unit
GET	/categories	List categories
POST	/categories	Create category
GET	/categories/{category}	Get category
PATCH	/categories/{category}	Update category
DELETE	/categories/{category}	Delete category

Method	Endpoint	Purpose
GET	/categories/tree	Get category tree
POST	/categories/{category}/activate	Activate category
POST	/categories/{category}/deactivate	Deactivate category
GET	/item-types	List item types
POST	/item-types	Create item type
GET	/item-types/{itemType}	Get item type
PATCH	/item-types/{itemType}	Update item type
POST	/item-types/{itemType}/activate	Activate item type
POST	/item-types/{itemType}/deactivate	Deactivate item type
GET	/attributes	List attributes
POST	/attributes	Create attribute
GET	/attributes/{attribute}	Get attribute
PATCH	/attributes/{attribute}	Update attribute
POST	/attributes/{attribute}/activate	Activate attribute
POST	/attributes/{attribute}/deactivate	Deactivate attribute
GET	/attributes/{attribute}/options	List options
POST	/attributes/{attribute}/options	Create option
GET	/attribute-options/{option}	Get option
PATCH	/attribute-options/{option}	Update option
DELETE	/attribute-options/{option}	Delete option
POST	/attribute-options/{option}/activate	Activate option
POST	/attribute-options/{option}/deactivate	Deactivate option
GET	/item-types/{itemType}/attributes	List assigned attributes
POST	/item-types/{itemType}/attributes	Assign attribute
PATCH	/item-types/{itemType}/attributes/{itemTypeAttribute}	Update assignment
DELETE	/item-types/{itemType}/attributes/{itemTypeAttribute}	Remove assignment

Method	Endpoint	Purpose
GET	<code>/item-types/{itemType}/attributes/metadata</code>	Get dynamic form metadata
GET	<code>/items</code>	List items
POST	<code>/items</code>	Create item
GET	<code>/items/{item}</code>	Get item
PATCH	<code>/items/{item}</code>	Update item
DELETE	<code>/items/{item}</code>	Archive item
POST	<code>/items/{item}/restore</code>	Restore item
POST	<code>/items/{item}/activate</code>	Activate item
POST	<code>/items/{item}/deactivate</code>	Deactivate item
GET	<code>/items/{item}/attributes</code>	Get item attributes
PATCH	<code>/items/{item}/attributes</code>	Update item attributes

13. Recommended Frontend Flow

When building the Angular item form:



▼
POST /items
|
▼
Item + dynamic attribute values

The frontend should **not hard-code the dynamic fields** for each item type. The metadata endpoint is the source of truth for which fields should be displayed and how they should behave.